WRITING BOOKS IN THE TERMINAL

The Inkhaven Manual

The Complete Guide to Writing a Book in the Terminal

Vladimir Ulogov

2026 · examples assume Inkhaven 3.0.0 or newer

Contents

About This Manual

Most writing tools ask you to leave the page. You open a browser to research, a second app to track your characters, a third to check continuity, a spreadsheet to count words, a design program to typeset. Inkhaven's premise is the opposite: **the whole of writing a book lives in one terminal window** — the prose, the world it is set in, the facts it must not contradict, the assistant you steer, and the finished PDF — and you never leave the keyboard to reach any of it.

This manual is the complete tour of that window. It assumes **no prior knowledge** — not of Inkhaven, not of the terminal beyond opening one — and it runs in reading order from installing the binary to publishing a book. Read it front to back once for the shape of the thing; return to any chapter when you reach the workflow it covers.

What Inkhaven is

Inkhaven is a single command-line program for writing books and long-form documentation. It pairs a full-screen editor for Typst — a modern typesetting language — with a local semantic index, an AI writing assistant you fully control, versioned snapshots, a backup pipeline, and an unusually deep set of **reading** intelligences that watch a manuscript for the things a long book gets wrong. Your prose lives as plain `.typ` files on disk; a local database tracks the hierarchy and the search index, but the words are always text you can read, diff, and version-control yourself.

> TERM **The manuscript is plain files**
>
> A paragraph is a `.typ` file on disk, inside a tree of folders for books, chapters, and subchapters. Inkhaven manages that tree and indexes it, but it never locks your words inside a proprietary format. You can open any paragraph in any editor, at any time, and Inkhaven will notice.

How this manual is organised

The book is in nine parts and a set of reference appendices:

The shape of the manual

```
  I    Getting Started ....... install, first project, the
panes
  II   Writing ............... the editor, the tree, snapshots,
search
  III  The AI Assistant ..... scopes, chat-with-your-book,
prompts, cost
  IV   The World & Facts .... places, characters, the world,
graph, timeline
  V    The Intelligences .... continuity, knowledge, read-
through, voices,
                              revision, the inner readers
  VI   Language & Verse ..... conlang, poetry, research,
scholarship
  VII  Producing the Book ... PDF, EPUB, web, technical docs
  VIII Scripting ............ the embedded Bund language
  IX   Keeping It Healthy ... backup, doctor, configuration
  A–D  Reference ............ keys, commands, config, the
feature index
```

Parts I through III are the ground floor — everything you need to write and get help. Parts IV and V are Inkhaven's distinctive depth: the machinery for a book with a world, a cast, and secrets that must stay kept. Parts VI through IX are the specialist tracks and the plumbing.

This manual, and the companions

Inkhaven ships a small library of **companion books**, each a deep dive on one domain — **Know Your Book** on the reading intelligences, **Building the World** on worldbuilding, **Poetry with Inkhaven**, **Grounding Your Book in Fact** on research, and others. This manual is deliberately **breadth-first**: it tells you what

each feature is, how to reach it, and how to run it, then points you to the companion when you want the full treatment.

FICTION

If you write **fiction**, the heart of the book is Parts IV and V — a world and cast Inkhaven can hold in mind, and the readers that watch continuity, knowledge, voice, and pacing across a whole manuscript.

NON-FICTION

If you write **non-fiction**, your centre of gravity is the facts and the graph (Part IV), research and sources (Part VI), and the technical-doc and index tools (Part VII) — the machinery of claims that must be right.

A word on the pictures

Inkhaven is a terminal application, so this manual illustrates it with **terminal frames** rather than photographs — a faithful rendering of what you will see on screen, set in a monospace type. A frame is truer than a screenshot for a text interface, and it keeps the book self-contained. When you see one, read it as "this is what the screen shows here."

WHAT YOU LEARNED

- Inkhaven is one terminal program for the **whole** of writing a book — prose, world, facts, an AI you steer, and the finished document.
- Your manuscript is always **plain `.typ` files** on disk; nothing is locked in a proprietary format.
- This manual is the **breadth-first tour**, install to publish; the companion books hold the deep dives.

PART I

Getting Started

CHAPTER 1

1

Installing Inkhaven

Inkhaven is a single program. When it is installed you have one command, `inkhaven`, and everything else in this book happens inside it or through it — the editor, the world, the facts, the assistant, the finished PDF. There is no server to run, no account to create, no companion app to keep open. This chapter gets that one command onto your machine, explains what happens the first time you run it, and tells you how to check that all is well before you write a word.

It assumes you can open a terminal and type into it, and nothing more. If you have never compiled a Rust program, that is fine — you will not have to understand Rust, only to install its toolchain and let it work for a few minutes. Read this chapter

once, top to bottom; you will not need it again until you set Inkhaven up on a second machine.

What you are installing

Inkhaven ships as one self-contained binary. It carries its own database engine, its own semantic-search index, and — after the first run — its own local embedding model. It does **not** depend on any external program being present on your system: no Node, no Python, no Docker, no database server, no paid AI account. The one genuinely optional companion is the Typst compiler, and only if you want to render your manuscript to PDF from inside the editor; everything else, including the plain-text and single-`.typ` exports, works with Inkhaven alone.

> TERM **Binary**
>
> A single executable file — here, one called `inkhaven`. "Installing" it means getting that file onto your machine and onto your `PATH` (the list of directories your shell searches for commands), so that typing `inkhaven` anywhere runs it. Everything Inkhaven does lives in that one file plus the per-user model cache it downloads once.

There are two ways to get the binary: let Cargo **compile** it for you (from crates.io or from source), or download a **prebuilt** one. Compiling is the path this chapter treats as canonical, because it always produces a binary matched to your exact machine and needs nothing but a Rust toolchain. The prebuilt paths are faster and are covered too.

Prerequisites

You need three things: a Rust toolchain, a terminal, and a little disk space. Only the first requires any action.

A Rust toolchain

Inkhaven is written in Rust, edition 2024, and requires a compiler of at least **version 1.85**. That minimum is not arbitrary — the 2024 edition and several language features Inkhaven relies on landed in that release. Anything newer is fine; Rust is strongly backward-compatible.

The official way to install Rust is

> **TERM rustup**
>
> the standard Rust toolchain installer — a small script that places `rustc` (the compiler) and `cargo` (Rust's build tool and package manager) under your home directory and adds them to your shell's `PATH`. It needs no administrator rights and installs entirely inside your own account.

. On macOS and Linux one line does it:

Installing the Rust toolchain via rustup

```
$ curl --proto '=https' --tlsv1.2 -sSf \
      https://sh.rustup.rs | sh
```

Accept the default when it prompts (option 1 is correct). When it finishes, either open a fresh terminal or reload your shell so that `cargo` is on the `PATH`:

Making cargo available in the current shell

```
$ source "$HOME/.cargo/env"
```

Then confirm both tools are present and new enough:

Verifying the toolchain

```
$ rustc --version
rustc 1.85.0 (a028ae42f 2026-01-09)
$ cargo --version
cargo 1.85.0 (d73d2caf9 2026-01-06)
```

If either prints "command not found", your shell has not yet picked up `$HOME/.cargo/bin` — close and reopen the terminal, or add that directory to your `PATH` by hand.

> **NOTE**
>
> The Rust toolchain is a one-time, whole-system install of roughly 600 MB. It is not part of Inkhaven and is reused by every Rust program you ever build. You install it once and forget it exists.

A terminal

Inkhaven is a terminal application. It runs in whatever terminal emulator you already have — the built-in Terminal on macOS, or any of the common Linux ones (GNOME Terminal, Konsole, Alacritty, kitty, WezTerm, xterm). It also runs perfectly over SSH, inside `tmux`, and on a tiling window manager, because it is text the whole way down and never reaches for a browser or a graphical window. Any terminal from the last decade with 256-colour support will do; the richer ones (kitty, iTerm2, WezTerm) additionally give you inline image previews, but those are a luxury, not a requirement.

Disk space

A fresh, complete install occupies a few hundred megabytes, most of it the one-time Rust toolchain. The parts that are actually Inkhaven are modest:

Disk footprint of a fresh install

```
  Rust toolchain ..... ~600 MB  one-time, shared by all Rust
  Inkhaven binary ..... ~90 MB   the compiled `inkhaven`
  Embedding model ..... ~120 MB  downloaded once, per-user
cache
  A new empty project . ~5 MB    database + index scaffolding
```

The embedding model is a single shared download, not a per-project cost: a hundred projects reuse the same cached model. A project's own footprint grows only with your prose and its snapshots.

Supported platforms

Inkhaven is released and supported on two platforms:

Supported release targets

```
Linux   x86_64-unknown-linux-gnu
macOS   aarch64-apple-darwin    (Apple Silicon)
```

These are the targets the release pipeline builds, tests, and ships prebuilt binaries for. Compiling from source with `cargo` works on any host Rust and the dependencies support — an Intel Mac, for instance, compiles cleanly even though there is no prebuilt asset for it — but the two above are the tested, first-class combination.

Why not Windows

Windows is **deferred**: there is no released Windows binary, and you should not expect one yet. The reason is specific and worth stating plainly, because it is not a matter of effort or interest.

Inkhaven computes its embeddings locally through `fastembed`, which in turn runs models through the ONNX Runtime by way of the `ort` crate. `ort` ships prebuilt runtime binaries for the Windows **MSVC** toolchain but not for the Windows **GNU** toolchain, and Inkhaven's build has historically targeted the GNU toolchain (alongside the DuckDB and font-handling C++ that MSYS2 builds cleanly). The build therefore fails at the ONNX Runtime bindings — upstream, in a dependency, not in Inkhaven's own code.

> **WHY IT IS DEFERRED, NOT ABANDONED**
>
> The fix is real and known; it is simply not on the critical path. Any one of three routes unblocks Windows: `ort` shipping a Windows-GNU prebuilt (the CI job is kept ready and would simply go green), adopting the Windows-MSVC target so that `ort`'s existing prebuilts apply, or bundling a build-time-verified runtime library in the release archive. The one route deliberately **rejected** is fetching and loading a runtime library at startup — that would run unsandboxed native code pulled over the network, a far larger attack surface than the model data Inkhaven does download, and against the project's no-external-binary principle. Until one of the sound routes is taken, Windows users should build under WSL (a Linux environment) and treat it as the Linux target.

Installing with Cargo

If you have the Rust toolchain, the shortest path to a working `inkhaven` is to let Cargo compile the published crate.

cargo install inkhaven — compile from crates.io

Inkhaven is published on crates.io; every release tag pushes a new version. This one command downloads the crate and its dependencies and compiles the lot:

Installing the published crate

```
$ cargo install inkhaven
```

Be patient the first time. The build pulls roughly a hundred crates and then compiles the heavy ones — DuckDB, `fastembed`, and the ONNX Runtime bindings are large C and C++ projects — so a first build takes on the order of **ten minutes** on a modern laptop. This is a one-off: Cargo caches the compiled dependencies, and you will not pay that cost again.

When it finishes, the binary lands in Cargo's binary directory, which is already on your `PATH` if rustup set your shell up:

Where cargo install puts the binary

```
~/.cargo/bin/inkhaven
```

That is the whole install. Skip ahead to "The first run" once `inkhaven --version` prints a version.

cargo binstall — the prebuilt fast path

If you would rather not wait for a compile and you have

> **TERM cargo-binstall**
>
> a small Cargo extension that downloads a **prebuilt** binary from a project's GitHub releases instead of compiling it. Install it once with `cargo install cargo-binstall`; thereafter `cargo binstall <crate>` fetches the right prebuilt asset for your platform.

installed, one command fetches the prebuilt binary and drops it into `~/.cargo/bin`:

Installing a prebuilt binary

```
$ cargo binstall inkhaven
```

`cargo-binstall` reads the packaging metadata from Inkhaven's `Cargo.toml`, picks the asset matching your platform off the GitHub releases, and installs it without compiling anything. This is the quickest route on the two supported platforms.

GitHub Releases — a direct download

You can also bypass Cargo entirely. Each release publishes a tarball per platform on the project's Releases page. Download the one for your platform, unpack it, and place the `inkhaven` binary anywhere on your `PATH`:

Installing from a release tarball

```
$ tar xzf inkhaven-<version>-<platform>.tar.gz
$ sudo install inkhaven /usr/local/bin/inkhaven
```

The releases live at github.com/vulogov/blackInkhaven/releases the asset names encode the version and the target triple so you can pick the right one at a glance.

cargo install --git — a specific tag or branch

When you want a particular tagged version, a pre-release branch, or your own fork, install straight from the git repository and name the tag:

Installing a specific tagged version

```
$ cargo install \
    --git https://github.com/vulogov/blackInkhaven \
    --tag v3.0.0
```

This compiles exactly the named revision. Drop `--tag` to build the default branch's current tip. Check the Releases page for the tag you actually want; the examples here name `v3.0.0`, the stable edition this manual documents.

Building from source

Compiling from a clone is the path to choose if you want to read or modify the code, track a development branch, or simply keep the whole thing under your own eye. It needs only the Rust toolchain and `git`.

First, clone the repository and enter it:

Cloning the source

```
$ git clone \
    https://github.com/vulogov/blackInkhaven.git
$ cd blackInkhaven
```

If you have no `git`, the GitHub page offers the same source as a zip; unpack it and `cd` into the folder. Then build an optimised binary:

Building the release binary

```
$ cargo build --release
```

The first build downloads about a hundred crates and compiles them; expect several minutes — commonly three to eight on a modern laptop, longer on older hardware because of the same heavy C/C++ dependencies noted above. Later builds are

incremental and reuse the cache, so they finish in seconds. When it is done, the binary is here:

Where the source build puts the binary

```
./target/release/inkhaven
```

You can run it straight from that path. To type just `inkhaven` from anywhere, copy it onto your `PATH`:

Putting inkhaven on your PATH

```
# system-wide (asks for your password)
$ sudo install target/release/inkhaven \
      /usr/local/bin/inkhaven

# or, no sudo, if ~/.local/bin is on PATH
$ cp target/release/inkhaven ~/.local/bin/
```

> **WHY THE --RELEASE FLAG MATTERS**
>
> Without `--release`, Cargo builds an unoptimised debug binary. It runs, but it is several times slower and a few hundred megabytes larger — embedding and search in particular feel sluggish. Always build `--release` for real writing; reserve the debug build for hacking on Inkhaven's own code.

Optional: the Typst compiler

Inkhaven writes and manages Typst, but it does not bundle the Typst compiler. If you want `inkhaven export pdf` (and the in-editor "build to PDF" chords) to actually produce a PDF, install Typst separately — it, too, is a single static binary on every platform, available from its own project page. Everything else, including the `export typst` path that emits one combined `.typ` file, works without it.

The first run

The install is inert until the first time Inkhaven opens a project. That first open does two one-time things worth understanding, so that a slightly longer startup does not alarm you.

The embedding model download

Inkhaven's semantic search — finding "the moment the lighthouse fails" even in a paragraph that never says **lighthouse** — is powered by a local embedding model. That model is not shipped inside the binary; it is downloaded the first time you initialise or open a project. The default is

> TERM **MultilingualE5Small**
>
> the default embedding model — a compact, multilingual model (English, Russian, German, French, Spanish, and more) about 120 MB in size. It is chosen as a sensible balance of quality against footprint; the configuration lets you switch to the Base or Large variant, at the cost of a larger download and slower embedding.

, roughly 120 MB, fetched once into a per-user cache and reused by every project thereafter.

The cache location depends on your platform:

Where the embedding model is cached

```
macOS  ~/Library/Caches/
           dev.inkhaven.inkhaven/embeddings/
Linux  $XDG_CACHE_HOME/inkhaven/embeddings/
           (defaults to ~/.cache/inkhaven/)
```

On a normal connection the download completes in well under two minutes, behind a splash screen that shows the elapsed time. It happens once per model: switching the configured model later downloads the new one on the next open and leaves the old one on disk until you remove the cache directory yourself. If you

ever change `embeddings.model` in a project's configuration, Inkhaven re-embeds every paragraph against the new model the next time it opens.

Model init and the project lock

After the model is present, Inkhaven initialises it into memory and takes an **advisory** lock on the project so that two sessions do not write the same database at once. The lock is a small file, `.inkhaven.lock`, in the project root, held for as long as the session runs and released automatically by the operating system when the process exits — even on a crash or a `kill -9`, so there is never a stale lock to clean up by hand.

> **THE LOCK INFORMS, IT NEVER BLOCKS**
>
> In keeping with Inkhaven's permissive character, opening a project that another session already holds does not fail. The launcher tells you who holds it — a process id, host, and time — and lets you open it anyway. Only genuine data-safety is at stake, and the choice stays yours. This is the same principle you will meet throughout Inkhaven: it warns, it does not forbid.

Once the model is cached and warm, subsequent launches are quick — there is no download and the lock is instantaneous.

Verifying the install

Two commands confirm a healthy install without touching any project. First, the version:

Confirming the binary runs

```
$ inkhaven --version
inkhaven 3.0.0
```

A version number means the binary is on your `PATH` and runs. If instead you see "command not found", the binary is not on your `PATH` — invoke it by its full path (`./target/release/inkhaven` from a source build, or `~/.cargo/bin/inkhaven` from a Cargo install), or copy it to a directory that is on your `PATH`.

Second, the help surface, which lists every top-level subcommand and the global flags:

Listing the command surface

```
$ inkhaven --help
TUI literary work editor for Typst books

Usage: inkhaven [OPTIONS] [COMMAND]

Commands:
  init      Initialize a new project
  add       Add a node to the hierarchy
  list      Print the hierarchy as a tree
  search    Run a semantic search
  export    Export the book(s)
  backup    Zip the whole project
  ...

Options:
  -p, --project <PROJECT>   Path to a project root
  -h, --help                Print help
  -V, --version             Print version
```

Any subcommand takes `--help` of its own — `inkhaven export --help`, `inkhaven init --help` — which prints that command's full set of flags. This is the authoritative reference for the exact surface of the version you have installed; when this book and the binary ever disagree, the binary's `--help` is right.

Troubleshooting the install

Most installs are uneventful. The handful of things that can go wrong have short answers.

command not found: inkhaven

Your shell cannot find the binary on its `PATH`. From a source build it lives at `./target/release/inkhaven`; from `cargo install` at `~/.cargo/bin/inkhaven`. Either call it by that full path, copy it into a directory that is on your `PATH` (`/usr/local/bin` or `~/.local/bin`), or alias it. Confirm

your `PATH` contains `$HOME/.cargo/bin` if you expected Cargo's install to be found automatically.

The first-run download is slow or stalls

The model download is the one part of startup that reaches the network. If the splash screen's elapsed counter climbs past a couple of minutes on a connection you know is working, something upstream is slow rather than broken. You can:

● ● ● *Recovering from a stalled first-run download*

```
- Check the machine actually has connectivity.
- Watch the splash's elapsed timer — under ~2 min
  is normal for MultilingualE5Small.
- Press Ctrl+Q to abort startup, then retry later.
```

Aborting is safe: nothing is half-written into the cache in a way that a later retry cannot replace.

Offline, air-gapped, or behind a proxy

Inkhaven never needs the network **except** for that one first-run model download (and, separately, for any external AI provider you choose to configure — which is entirely optional). Two consequences follow.

On a **proxy**, make sure the standard `HTTPS_PROXY` environment variable is set before the first run so the download can reach out. On a genuinely **air-gapped** machine, you cannot download at all — so seed the cache from a networked machine instead: install and run Inkhaven once on a connected computer, then copy the populated cache directory (the platform paths listed under "The first run") onto the air-gapped machine before its first launch. With the model already present, Inkhaven starts fully offline and stays offline for the whole of writing, worldbuilding, search, and snapshots — every subsystem but the external providers is on-device.

A configuration error on open

If opening a project reports a missing configuration field, you are opening a project whose `inkhaven.hjson` was written by an older release than the binary you just installed. Either add the missing field by hand, or regenerate a fresh configuration from the current template with `inkhaven init --force` (back the old file up first). New projects created by the version you installed never hit this.

The CLI and the TUI

One binary, two shapes. Understanding the split now will save confusion in every later chapter, because this manual moves freely between them.

Run `inkhaven` with **no subcommand** — inside a project directory, or with a `--project` path — and it launches the full-screen editor: the

> **TERM TUI**
>
> the Text User Interface — Inkhaven's full-screen, keyboard-driven editor, with its tree, editor, search, AI, and output panes. It takes over the terminal, and you leave it with `Ctrl+Q`. This is where you actually write.

, the interactive program that fills the terminal and is the subject of most of this book:

Opening the editor

```
$ cd ~/Books/my-novel
$ inkhaven                 # opens the TUI here
# …or from anywhere:
$ inkhaven --project ~/Books/my-novel
```

Run `inkhaven` **with** a subcommand and it does one job and exits, printing to the terminal without ever entering full-screen mode. These headless commands are what you script, pipe, and run in continuous integration:

Headless, one-shot commands

```
$ inkhaven init ~/Books/my-novel
$ inkhaven --project ~/Books/my-novel list
$ inkhaven --project ~/Books/my-novel \
      search "the lighthouse fails"
$ inkhaven --project ~/Books/my-novel \
      export pdf --status ready
```

The global `--project` flag (also `-p`, or the long alias `--project-directory`) tells any command which project to act on, and defaults to the current directory — so `cd` into a project first and you can drop it entirely. The top-level command families you will meet across the book group roughly like this:

init · add · mv	Create and shape the hierarchy
list · outline	Inspect the tree from the shell
search · book-rag	Semantic search and retrieval
export · index	Produce PDF, Typst, EPUB, indexes
backup · restore	Project-level disaster recovery
reindex · doctor	Reconcile store with disk; health
ai	One-shot inference from the shell

A few subcommands — the standalone configuration editor, the research and worldbuilder and linguistic workspaces — are themselves full-screen TUIs rather than one-shot commands; the book flags each where it introduces it. Everything else you type after `inkhaven` runs, prints, and returns you to your prompt.

FICTION

If you write **fiction**, your usual entry is the bare `inkhaven` — you live in the editor, with the tree, the world, and the readers a keystroke away. The headless commands matter mainly at the edges: `init` to begin, `backup` to be safe, `export` to ship.

NON-FICTION

If you write **non-fiction** or documentation, you will lean on the headless side far more — `search`, `export`, `index`, and the check commands slot naturally into build scripts and continuous integration, where an editor cannot go.

With `inkhaven` installed, verified, and its two shapes clear, you are ready to create a project — which is where the next chapter begins.

WHAT YOU LEARNED

- Inkhaven is **one self-contained binary**; the only prerequisite you must install yourself is a **Rust toolchain of version 1.85 or newer** via rustup.
- Install it by compiling — `cargo install inkhaven` (a 10-minute first build) or `cargo build --release` from a clone — or grab a prebuilt binary with `cargo binstall` or from GitHub Releases.
- The supported targets are **Linux x86_64** and **macOS Apple Silicon**; Windows is deferred because the ONNX Runtime bindings `fastembed` needs have no Windows-GNU prebuilt.
- The **first run** downloads a 120 MB embedding model once into a per-user cache, then takes an advisory project lock that **informs but never blocks**.
- Verify with `inkhaven --version` and `inkhaven --help`; running `inkhaven` bare opens the **TUI**, while `inkhaven <subcommand>` runs **headless** and exits.

CHAPTER 2

2

Your First Project

A book, to Inkhaven, is a **directory**. Not a file with an opaque extension, not a row in some cloud database you cannot see — a plain folder on your own disk, with your prose in readable text files and a small local database beside them that remembers the shape of the thing. This chapter creates one, opens it up, and names every part, so that when a later chapter says "the Facts book" or "the `.inkhaven/` sidecars" or "a structural leaf" you already know exactly where on disk it lives and what it is for.

We will run one command, read what it laid down, meet the twenty **system books** Inkhaven seeds into every project, and then make a book, a chapter, and a

paragraph of your own. By the end you will have a project you can open, close, back up, and put under version control — and a mental map of every file in it.

The one command that starts everything

Everything begins with `inkhaven init`. It takes a single positional argument — the directory the project should live in — and creates it if it does not yet exist. The convention is one project per book (or per tightly-related set), kept somewhere like `~/Books/`.

Creating a project

```
$ inkhaven init ~/Books/my-first-project
Initialized inkhaven project at /home/you/Books/my-first-
project
  config:    …/my-first-project/inkhaven.hjson
  prompts:   …/my-first-project/prompts.hjson
  store db:  …/my-first-project/metadata.db
  vecstore:  …/my-first-project/vectors
  books:     …/my-first-project/books
```

The directory need not exist beforehand — Inkhaven makes it. If it **does** exist, Inkhaven refuses to touch it silently, because `init` starts from a clean database and re-initialising means **wiping** what is there. It asks first:

The overwrite guard

```
$ inkhaven init ~/Books/my-first-project
Directory `/home/you/Books/my-first-project` already exists.
Remove it and re-initialise? [y/N]
```

Only `y` or `yes` proceeds; a bare Enter, an `n`, or end-of-input all abort with your files left untouched. There is one further safety interlock: Inkhaven will **refuse** to wipe a directory that your current shell is sitting inside, because deleting the ground beneath your own feet leaves the terminal in a broken state. `cd` out first.

The two flags

`init` has exactly two options beyond the path.

--force — Skip the [y/N] prompt and overwrite an existing directory unattended. For scripts and fresh-checkout automation — never needed interactively.

--template <name> — Scaffold a starting tree instead of an empty one. Defaults to empty.

The templates are pre-built manuscript skeletons applied **after** the standard init, so a template never changes what init itself lays down — it only adds books and chapters on top. Run `inkhaven template list` to see them all; the built-in set is:

inkhaven init --template <name>

```
empty          the default — system books only, no manuscript
novel          three-act manuscript + Characters stubs
nonfiction     intro / parts / conclusion + Research method
rpg-sourcebook Setting/Rules/Adventures + Places/Artefacts/
Threads
technical      Overview / Reference / Tutorials / Index
nanowrimo      like novel, with a 50 000-word goal + pacing
```

If you are unsure, take `empty`. You can build the same tree by hand in the TUI in a minute, and this chapter shows you how. A template is a convenience, never a commitment.

> **FIRST RUN DOWNLOADS A MODEL**
>
> The very first `init` on a machine fetches a multilingual embedding model (about 120 MB) into a **per-user** cache — not into the project. On macOS that is `~/Library/Caches/dev.inkhaven.inkhaven/embeddings/`; on Linux `~/.cache/inkhaven/embeddings/`; on Windows under `%LOCALAPPDATA%`. Every later `init` reuses it, so only the first project pays the download. This is the model that powers semantic search and the reading intelligences — all of it runs on your own machine, offline.

What init writes to disk

`cd` into the new project and list it. Ignoring the model cache (which lives elsewhere), a fresh project is small and entirely legible:

● ● ● *A fresh project, top level*

```
my-first-project/
├── inkhaven.hjson      the configuration file
├── prompts.hjson       your prompt library
├── metadata.db         hierarchy + node metadata (DuckDB)
├── blobs.db            paragraph bodies, snapshots, blobs
├── frequency.db        the full-text search index
├── vectors/            the HNSW semantic-search index
└── books/              your prose — one folder per book
```

Two of these you will edit by hand; the rest Inkhaven owns. The two yours are `inkhaven.hjson` (theme, language, AI provider, autosave cadence — Chapter and appendix on configuration cover every field) and `prompts.hjson` (your library of reusable AI prompts). The four database entries — `metadata.db`, `blobs.db`, `frequency.db`, and the `vectors/` directory — are the local store, created and maintained by the embedded database engine. You never open them with a text editor.

> TERM **The store is three databases and an index**
>
> Inkhaven's metadata lives in DuckDB files beside your prose. `metadata.db` holds the **hierarchy** — every book, chapter, and paragraph as a node with a stable UUID, its title, order, and file path. `blobs.db` holds paragraph bodies and versioned snapshots. `frequency.db` is the full-text index behind keyword search. `vectors/` is the HNSW index that makes **semantic** search and the reading intelligences possible. Your words themselves, though, are never trapped in there — they are the `.typ` files under `books/`, and the database merely points at them.

This split is the heart of Inkhaven's data model, and worth dwelling on for a moment. The **prose is the filesystem**. The **database is an index over the**

filesystem. If the two ever drift apart — you edited a paragraph in another editor, or a crash left them out of step — `inkhaven reindex` re-reads the `.typ` tree and rebuilds the database from it. The files are the source of truth; the database is derived, and therefore always rebuildable. That is why your manuscript is safe even if you never trust the database at all: it is ordinary text, in ordinary folders, that you can read, `grep`, diff, and commit to Git without Inkhaven's help.

Where session, backups, and caches live

Beyond the store, Inkhaven keeps a handful of **dotfiles** and one **dot-directory** in the project root. None of these exist right after `init` — they appear as you use the project — but knowing them now saves confusion later.

The runtime sidecars (appear with use)

```
my-first-project/
├── .session.json           cursor + which paragraph was open
├── .inkhaven-backup.json   timestamp of the last backup
├── .inkhaven.log           the runtime log (append-only)
└── .inkhaven/              advisory sidecars + local caches
    ├── drift.json          continuity-drift findings
    ├── facts_scan.json     fact-check results
    ├── tensions.json       pacing / tension curve
    ├── continuity.json     the continuity bible cache
    ├── submissions.json    submission-package tracker
    ├── ai_usage.json       rolling AI cost ledger
    ├── prose.duckdb        per-chapter prose metrics
    ├── voices/             character-voice profiles
    └── digest-<slug>.json  per-book cached digests
```

The three top-level dotfiles are small and transient. `.session.json` remembers which paragraph you had open and where the cursor sat, so re-opening the project puts you back exactly where you left off. `.inkhaven-backup.json` is a single timestamp the auto-backup logic checks against your configured `backup.max_age`. `.inkhaven.log` is the append-only log — the first place to look when something misbehaves.

The `.inkhaven/` directory is a growing collection of **advisory sidecars**: the cached output of scans and the reading intelligences. Every file in it is disposable. Each records what some analysis last found — drift, fact-checks, tension curves, voice profiles — so that opening a view is instant instead of re-computing from

scratch. Delete any of them and Inkhaven simply recomputes on next use. Crucially, **nothing here is your prose**: these are derived opinions about your prose, kept to one side. Chapters throughout Parts IV and V describe each sidecar where its feature is covered; for now, treat `.inkhaven/` as "the scratchpad the intelligences share."

> **FOR VERSION CONTROL**
>
> Commit `inkhaven.hjson`, `prompts.hjson`, and `books/` — the prose and its configuration. You may commit the `.db` files too (they are your index and snapshots), but they are large and rebuildable, so many authors add `*.db`, `vectors/`, `.inkhaven/`, `.session.json`, and `.inkhaven.log` to `.gitignore` and let `inkhaven reindex` rebuild the store on a fresh clone.

The anatomy of the manuscript tree

Open `books/` and you find the real structure. Inkhaven's hierarchy — **Book → Chapter → Subchapter → Paragraph** — is mirrored one-to-one onto the filesystem: branches are folders, leaves are files, and every entry carries a zero-padded numeric prefix so a plain directory listing sorts in reading order.

One book on disk

```
books/
└── my-first-book/
    ├── 01-chapter-one/
    │   ├── 01-the-storm.typ
    │   ├── 02-landfall.typ
    │   └── 02-a-quiet-town/          ← a subchapter
    │       ├── 01-the-inn.typ
    │       └── 02-the-innkeeper.typ
    └── 02-chapter-two/
        └── 01-morning.typ
```

Read that tree carefully, because its rules are the whole model:

- A **book** is a folder named by its bare slug (`my-first-book`) — no number, because books are ordered among themselves in the database, and their folders sit at the top level of `books/`.
- A **chapter** and a **subchapter** are numbered folders (`01-chapter-one`, `02-a-quiet-town`). A subchapter is simply a chapter-level folder that lives inside a chapter — the same kind of container, one level down.
- A **paragraph** is a numbered `.typ` **file** (`01-the-storm.typ`). This is the leaf: the actual unit of prose you edit. Despite the name, a "paragraph" can hold as much text as you like — think of it as a titled **section** of the manuscript, the smallest thing Inkhaven versions, embeds, and re-orders.

The numeric prefixes are Inkhaven's bookkeeping, not yours to hand-edit. When you reorder rows in the tree, Inkhaven renumbers the files on disk to match; the slug (the human-readable part after the number) comes from the title, and a paragraph with an empty title gets its slug — and title — from the first sentence you type on first save.

> TERM **The manuscript is a Typst project in disguise**
>
> Each paragraph file is a fragment of Typst, the typesetting language. A new paragraph opens with a single `=` heading line — Typst's level-one heading marker — so the paragraph renders as its own section in the final book. When you export, Inkhaven walks the tree in order, assembles the fragments into one Typst document, and hands it to the typesetter. Nothing about the on-disk format is proprietary: it is Typst, all the way down.

The node model at a glance

Every row in the tree is a **node**, and every node has a **kind**. Four kinds are the structural spine you have already met — Book, Chapter, Subchapter, Paragraph. Alongside the prose paragraph, though, Inkhaven allows several **non-prose leaves**: siblings in the same tree that carry something other than Typst prose. Each has its own on-disk extension and its own chapter later in this manual; here they are only named, so you recognise them in a tree.

● ● ● *Leaf kinds at a glance*

```
KIND          ON DISK        WHAT IT HOLDS
paragraph     NN-slug.typ    prose (Typst) — the default leaf
image         NN-slug.png    a picture: png / jpg / webp / svg
hjson  ¶      NN-slug.hjson  structured data (a place, a
source…)
jinja  ⍰       NN-slug.jinja  a template rendered to Typst on
export
bund   ⚙      NN-slug.bund   an embedded Bund script
```

A few notes to fix the picture:

- An **image** is a first-class node, not a paragraph. Drop a `.png` / `.jpg` / `.webp` / `.svg` into the tree and Inkhaven ships the bytes into the assembled book with the right image call; a caption and alt-text ride along in the metadata.
- An **hjson** leaf is a paragraph whose content type is data rather than prose. This is how the world books store structure — a place, a character, a bibliography entry — as an HJSON record instead of paragraphs of text.
- A **jinja** leaf is a template: it holds Jinja markup that Inkhaven renders down to Typst at assembly time, for repeated or generated structure.
- A **bund** leaf is a script in Inkhaven's embedded Bund language, evaluated into the scripting engine when the project opens — the home of custom hooks and rules. Its default home is the Scripts system book, but it can live anywhere.

There is one more flavour that is **not** a separate file type but a **subtype** of an ordinary Typst paragraph — the **structural paragraph**. In the editor, the structural-subtype picker turns a paragraph into a piece of recognised scaffolding — code block, admonition, mathematics, procedure, or table — each with its own glyph in the tree (⌨ ⚠ ∫ ≡ ⊞). It is still a `.typ` file; the subtype only tells Inkhaven how to treat and render it. A single leaf can be cycled through its types — `typst → hjson → jinja → bund` — as your needs change. Every one of these gets a full treatment in a later chapter; for now, the point is simply that **the tree holds more than prose**, and each non-prose leaf is still a plain, readable file.

The system books

Here is the part that surprises people opening a fresh project: it is not empty. Before you have written a word, `inkhaven list` shows a stack of books already there. These are the **system books** — reference and machinery books Inkhaven seeds into **every** project, on every open, whether you use them or not. They carry stable internal tags, they are marked **protected** (you cannot delete or rename them), and anything you create lands **above** them, at the top of the tree.

> **HOW MANY, REALLY**
>
> Older documentation counted "six" and then "nine" system books; the project has grown since. The authoritative list is the `SYSTEM_BOOKS` table in the source, and as of this edition it seeds **twenty**. If a future release adds one, it appears automatically the next time you open any existing project — the seeder is idempotent and back-fills.

They divide cleanly into three groups: reference books you write into, world books that hold your fiction's ground truth, and machinery books Inkhaven and its tools own. First, the reference and prose books:

System books · reference & prose

```
Notes        free-form scratch prose, kept out of the
manuscript
Research     research notes and gathered source material
Sources      bibliography entries (HJSON) → compiled to a .bib
Glossary     canonical terms + banned synonyms (terminology)
Snippets     reusable Typst blocks, #include'd elsewhere
Prompts      your AI prompt library (seeded with examples)
Help         Inkhaven's own manual — F1 answers by RAG here
```

Then the **world** books — the ground truth a work of fiction must stay consistent with. These are where Parts IV and V of this manual do their work:

System books · the world

```
Facts        the world's invariants — the fact-check reference
Places       your gazetteer — locations, named and highlighted
Characters   the cast — names light up in the editor
Artefacts    objects of significance — items, relics, devices
World         simulation output (the realworld compiler writes
here)
Threads      narrative plot threads and their arc shapes
Planning     story structure — beats and acts of a framework
Mythology    declared symbols, motifs, and archetypes
Intent       your declared authorial choices (silences
findings)
```

And finally the **machinery** books — homes for the tooling that surrounds the prose:

System books · machinery

```
Typst        reusable Typst templates and skeletons
Scripts      .bund scripts auto-loaded into the engine at open
Language     invented-language books (one child book per
conlang)
Submissions  the submission package — query letter, synopsis…
```

A handful of these deserve a sentence of **why** now, because you will meet them again and again:

- **Notes** and **Research** are the free-form catch-alls — prose that supports the book without being in it. The AI can be scoped to read them.
- **Facts** is the reference the continuity and fact-checking tools ground against: the climate, the geography, the chronology your chapters must not contradict. **Grounding Your Book in Fact** is the companion book for it.
- **Places**, **Characters**, and **Artefacts** are **lexicon** books — Inkhaven highlights their names where they appear in your prose, and you can ask the AI about any of them by name.
- **Prompts** ships pre-seeded with `.example` prompts for the built-in AI actions; rename one to drop the `.example` suffix and it overrides the default. **Help** is

Inkhaven's own documentation, which is why pressing `F1` can answer questions about the program itself.
- **Intent** is where you record "I meant to do that" — a declared choice that tells the reading intelligences to stop flagging something they would otherwise report.

You do not need to touch most of these on day one. They are there so that when a later feature needs a home — a bibliography, a cast list, a plot thread — it already has one, in a known place, tagged the same in every project you will ever open.

Opening the project and making your first book

You have a project on disk. Now open it. Running `inkhaven` with no arguments opens the project in the **current** directory; `--project <path>` opens one elsewhere.

Opening the editor

```
$ cd ~/Books/my-first-project
$ inkhaven
# — or, from anywhere —
$ inkhaven --project ~/Books/my-first-project
```

The full-screen editor comes up in a multi-pane layout — a search bar across the top, the **Tree** on the left, the **Editor** in the middle, an **AI** pane on the right, and a status line along the bottom. Chapter three walks the panes in detail; here we only need the Tree, which has focus on startup, and where you see the system books stacked and waiting.

The Tree is driven by single letters. To build the spine of a book you need just three of them — one for each level — plus Enter to open a leaf for editing.

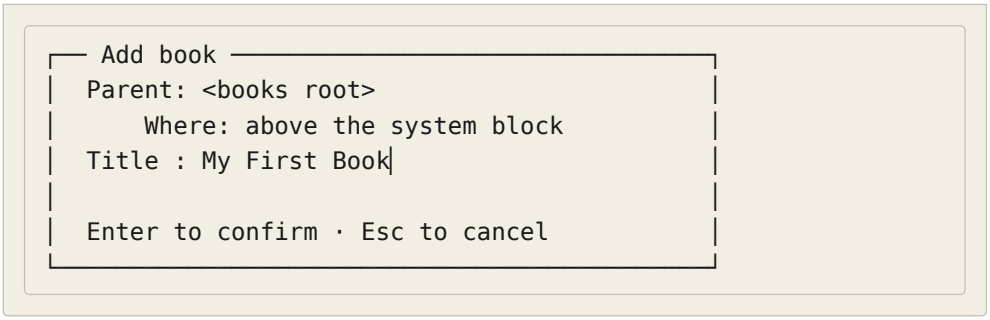
B	add a book at the root (above the system books)
C	append a chapter under the selected book
A	append a subchapter under the selected chapter
+	append a paragraph under the selected branch
Enter	open the selected paragraph in the Editor

F2 rename the selected node

Each of `C`, `A`, and `+` has an **insert-after** twin — `V`, `S`, and `P` respectively — that drops the new node immediately below the current row rather than at the end of its parent. You will reach for those when a manuscript grows and you need to slot something in the middle; for a first pass, append is all you want.

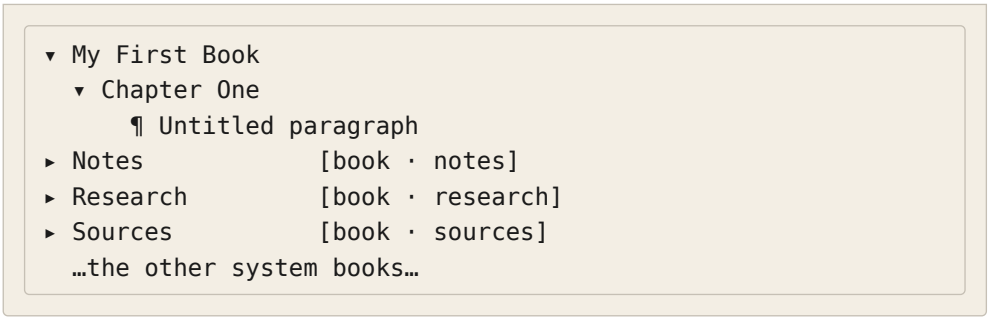
Press `B`. A small dialog asks for the title; type one and press Enter, and your book appears at the very top of the tree, above `Notes`:

Adding a book



With the cursor on your new book, press `C` to add a chapter beneath it; with the cursor on the chapter, press `+` to add a paragraph. You may leave the paragraph's title blank — Inkhaven will name it from your first sentence when you save. The tree now shows your work sitting above the seeded books:

Your first structure



Put the cursor on the paragraph row and press Enter. Focus moves to the Editor, which shows the paragraph's starting template — a single Typst heading line:

A new paragraph in the Editor

```
= Untitled paragraph

```

Press `End`, then Enter twice to leave the heading behind, and write. The border around the Editor turns **yellow** the moment the buffer is dirty. Press `Ctrl+S` to save: the `.typ` file is written under `books/`, the metadata is updated, and a fresh embedding is computed for search — and the border returns to **green**. Because you left the title empty, the Tree row now reads back your opening sentence, and the file on disk has been renamed to match its new slug.

That is a complete round trip: a book, a chapter, a paragraph, saved to disk as plain Typst, indexed for search, and ready to reopen exactly where you left it. To leave, press `Ctrl+Q` — Inkhaven autosaves anything dirty and records the session so your next launch reopens this very paragraph.

Two ways to shape the tree first

Before you pour in words, it is worth a minute's thought about **shape**, because the tree you build is the outline you will write against — and a novelist and a non-fiction author reach for it differently.

FICTION

Start with the **spine of the story**, not the prose. Make one book, then a chapter per act or major movement, and let paragraphs be **scenes**. Lean on the world books early: drop your cast into **Characters** and your locations into **Places** as you invent them, so their names highlight in the prose and the continuity readers have something to watch. The `novel` template lays exactly this down if you would rather not start empty.

NON-FICTION

Start from the **table of contents**. Make a book, a chapter per part or major section, and subchapters for the sections beneath — the tree **is** your outline, and it will become the document's structure verbatim. Put your gathered material and citations into **Research** and **Sources** from the first day, so claims have a home and a bibliography assembles itself. The `nonfiction` and `technical` templates scaffold this shape.

Neither choice is binding. The tree reorders freely — rows move up and down, paragraphs move between chapters, whole branches relocate — and every move renumbers the files on disk for you. Build the shape you can see today; reshape it the moment the book tells you to.

WHAT YOU LEARNED

- `inkhaven init <dir>` creates a project directory; `--force` skips the overwrite prompt and `--template` scaffolds a starting tree, defaulting to **empty**. The first ever init downloads a shared embedding model to a per-user cache, not into the project.
- A project is **plain files**: `inkhaven.hjson` and `prompts.hjson` are yours to edit; `metadata.db`, `blobs.db`, `frequency.db`, and `vectors/` are the local store; `books/` is your prose. The files are the source of truth and the store is a rebuildable index — `inkhaven reindex` restores it from the tree.
- The hierarchy **Book → Chapter → Subchapter → Paragraph** is mirrored onto disk as folders and numbered `.typ` files; alongside prose paragraphs the tree can hold **image**, **hjson**, **jinja**, and **bund** leaves, plus structural-paragraph subtypes.
- Every project is seeded with **twenty protected system books** — reference (Notes, Research, Sources, Glossary, Snippets, Prompts, Help), world (Facts, Places, Characters, Artefacts, World, Threads, Planning, Mythology, Intent), and machinery (Typst, Scripts, Language, Submissions) — that you cannot delete and that your own books sit above.
- In the TUI the Tree is driven by single letters — `B` book, `C` chapter, `A` subchapter, `+` paragraph, Enter to edit, `Ctrl+S` to save — and the `.inkhaven/` directory plus the `.session.json` / `.inkhaven-backup.json` / `.inkhaven.log` dotfiles hold disposable session, backup, and cache state.

CHAPTER 3

3

The Four Panes

Everything in the last two chapters happened somewhere on one screen. Now we name the places. Inkhaven fills your terminal edge to edge with a single full-screen window, and the whole of writing a book takes place inside it — you never open a second app, never reach for a mouse you do not need, never lose your place. This chapter is the map of that window: the four regions it is divided into, how you move your attention between them, and the two-key **chord** language that reaches every command the tool has. Learn this chapter and the rest of the manual becomes a matter of looking things up; skip it and every later chapter will feel like a room you entered through the wrong door.

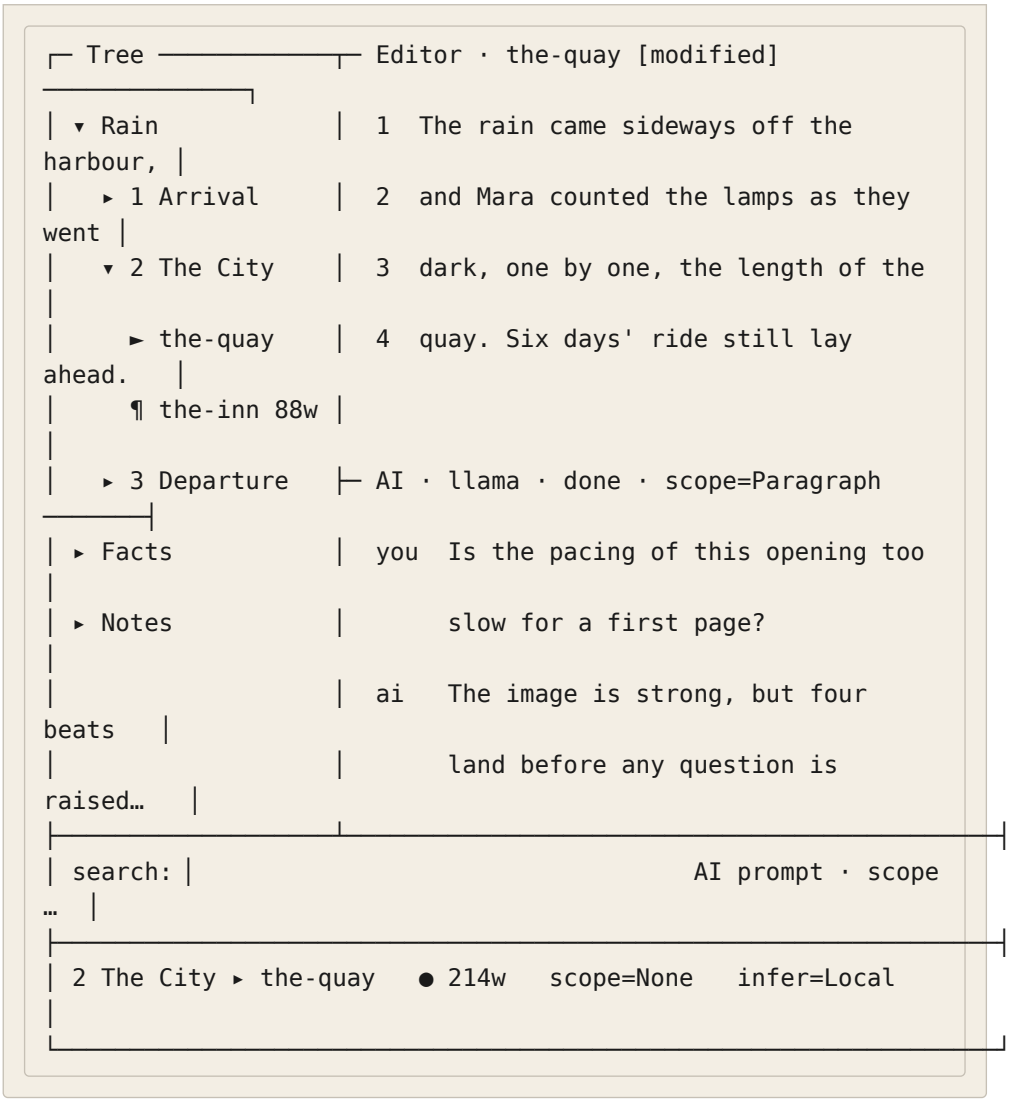
One window, four regions

Open a project and the terminal divides into four working areas. On the left, a narrow **Tree** holds the shape of your book — its parts, chapters, and paragraphs, folded and unfolded like an outline. Filling the centre is the **Editor**, where the open paragraph's words live. To its right sits a region that shows one of three things at a time — most often the **AI** pane, a conversation with your assistant. Across the bottom run two thin input bars — a Search bar and the AI prompt — and beneath them a single **status bar** summing up the moment. That is the whole instrument.

> TERM **Pane**
>
> A **pane** is one of the window's working regions — the Tree, the Editor, the AI pane, and the others. Exactly one pane has **focus** at any moment: the pane your keystrokes are talking to. Everything you type is read as input to the focused pane (or, when a chord is in flight, as a command).

The book you are reading calls this "the four panes" for the shape you meet first — Tree, Editor, AI, and the region beside the Editor — but the window is really a small family of surfaces, and the right-hand region alone can show three different panes. Hold the four in mind as the frame; the rest hang off it.

● ● ● *The whole window — Tree, Editor, and the right region*



Read it top to bottom. The Tree names the book; the green ► marks the paragraph currently loaded in the Editor, wherever your tree cursor happens to be. The Editor holds that paragraph's lines with a number gutter down the left. The right region — here showing the AI pane — carries the conversation. The two bottom bars wait for a search query and an AI prompt. The status bar, last of all, tells you where you stand: the breadcrumb to the open paragraph, its word count, the red ● that means unsaved edits, and the current AI scope and inference mode.

TERMINAL-FIRST

Because it is only text, this window runs anywhere a terminal does — over SSH, inside `tmux`, on a bare tiling window manager, on a laptop on a train with the lid half-closed. There is no separate GUI, no browser tab, nothing to install beyond the one binary. The frames in this book are faithful to what you will actually see.

The right-hand region — three panes in one place

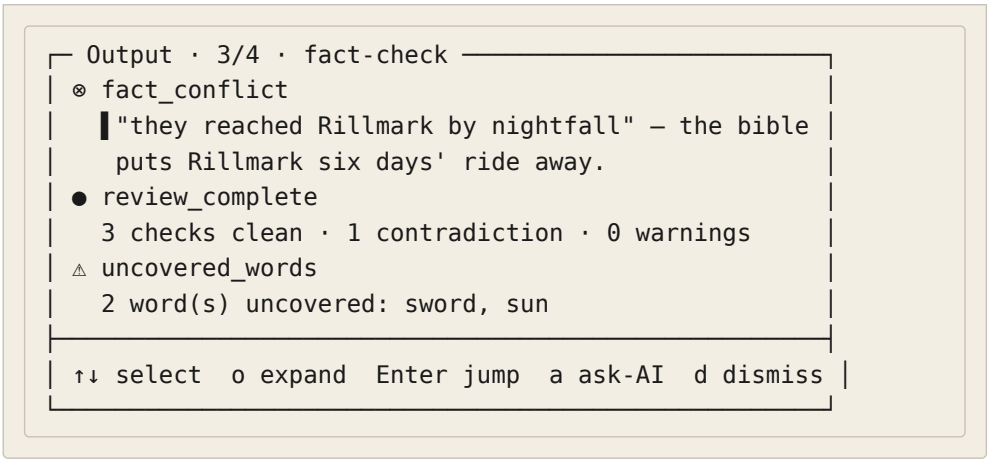
The region to the right of the Editor is not a single pane but a slot that holds **one of three** at a time. This is the part of the layout newcomers find surprising, so it is worth naming plainly.

TERM **The right region**

The right-hand slot cycles between three panes: the **AI pane** (a conversation you drive), the **Output pane** (a notice board subsystems post to), and the **Thoughts pane** (a quiet home for long reflective text). Only one is visible at a time; switching between them is a single chord.

The three are three different **kinds** of surface, which is why they are three panes and not one. The AI pane is a **dialogue** — you ask, the model answers, the history scrolls. The Output pane is a **notice board** — structured, one-way messages that the fact-checker, the continuity watch, the translation engine, a finished background job, or a Bund script post for you to read, act on, and dismiss; each carries a severity glyph (● info, ⚠ warning, ⊗ contradiction, ↻ still-running) and expires on its own so the board never becomes a junk drawer. The Thoughts pane is a **reading** surface — a scrollable, read-only place where the longer, essayistic output lands, such as the Inner Theologian's questions or a developmental brief.

● ● ● *The right region showing the Output pane — a notice board*



Which pane the slot shows when you launch is remembered: on a project's very first start it opens on the Output pane, and after that Inkhaven restores whichever of the three you last left active, exactly as it restores which paragraph was open. When fresh content arrives for a pane it will quietly surface that pane for you — unless you are actively working in a right pane at the time, in which case it holds still rather than steal your place mid-read.

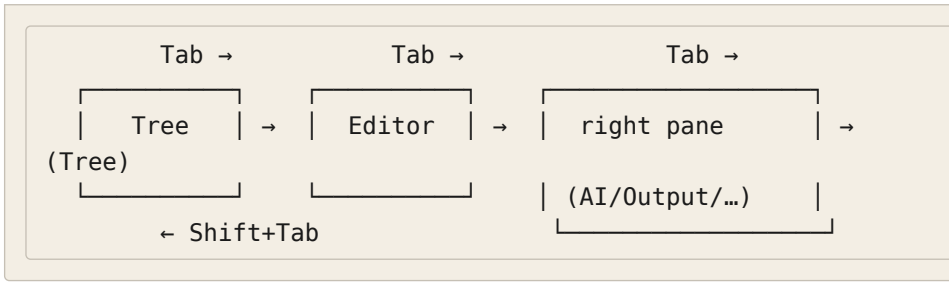
Moving your attention — focus and navigation

Only one pane listens to your keys at a time. Moving that attention around is the most common thing you will do, so Inkhaven gives you both a cycle and a set of direct jumps.

The plain cycle — Tab

The everyday key is `Tab`. It walks focus through a short ring — the Tree, the Editor, and **whichever right pane is currently shown** — then back to the Tree. It is a three-stop loop, not a tour of every surface: the right region counts as one stop, so if the slot is on the Output pane, `Tab` visits the Output pane; if it is on the AI pane, `Tab` visits the AI pane. `Shift+Tab` walks the same ring in reverse. You will reach for `Tab` far more than any other navigation key, and because Inkhaven intercepts it before the text editor sees it, pressing `Tab` never inserts a literal tab into your prose.

Tab cycles a three-stop ring



Cycling the right region — Ctrl+B Tab

To change **which** pane the right slot shows, you cycle the region itself with `Ctrl+B Tab`. Each press advances the slot through the three panes in order — Output, then AI, then Thoughts — and moves focus onto it; `Ctrl+B Shift+Tab` steps backward. This is a different motion from plain `Tab`: `Tab` moves your focus **to** the region, `Ctrl+B Tab` changes **what the region is**. The chord works from anywhere — even mid-edit — because the `Ctrl+B` prefix (which you will meet properly in a moment) swallows the keystroke before the editor could read `Tab` as a tab.

Jumping straight to a pane

When you know exactly where you want to be, skip the cycle and jump. Each of the main surfaces has a direct key, so no pane is ever more than one chord away.

Tab	Cycle focus: Tree ⟶ Editor ⟶ current right pane ⟶ Tree.
Shift+Tab	Cycle focus in reverse.
Ctrl+B Tab	Cycle the right region Output ⟶ AI ⟶ Thoughts, and focus it.
Ctrl+B Shift+Tab	Cycle the right region backward.
Ctrl+T	Focus the Tree pane.
Ctrl+1	Focus the Editor pane.
Ctrl+3	Focus the AI pane.
Ctrl+/	Focus the Search bar (top). Ctrl+4 does the same.
Ctrl+I	Focus the AI prompt bar (bottom). Ctrl+5 does the same.

How you know which pane has focus

Focus is never a guess — every pane shows it. The Editor's border carries the signal most vividly: **green** when the pane is focused and saved, **yellow** when focused with unsaved edits, and plain **white** when it is not focused at all. In the Tree, the open paragraph keeps its green `►` marker whether or not the Tree has focus, so you can always find your place. And when you have started a chord but not finished it, a yellow **META** chip lights up in the status bar to tell you the tool is waiting for your next key. Between the border, the marker, and the chip, the window always tells you what it is listening to.

> FOCUS-LOSS AUTOSAVE
>
> Whenever focus leaves the Editor — by `Tab`, by a direct jump, by `Esc` from another input — the open paragraph is saved for you if it was dirty. You can shift your attention mid-sentence without a thought for losing work; the words are on disk before the next pane has your keystrokes.

The chord system — the heart of the interface

Inkhaven has more commands than a keyboard has keys, and it refuses to bury the useful ones behind menus. Its answer is the **chord**: a short, deliberate two-key sequence, led by a prefix that tells the tool "a command is coming." Once the idea clicks, the whole interface opens up, because every serious action in the program is one chord away from wherever you are.

> TERM **Chord**
>
> A **chord** is a two-key command: a **prefix** key, then an **action** key. You press the prefix, release it, then press the second key — they are struck in sequence, not held down together. Between the two presses the tool is in a brief **pending** state, waiting for the action key and showing you it is waiting.

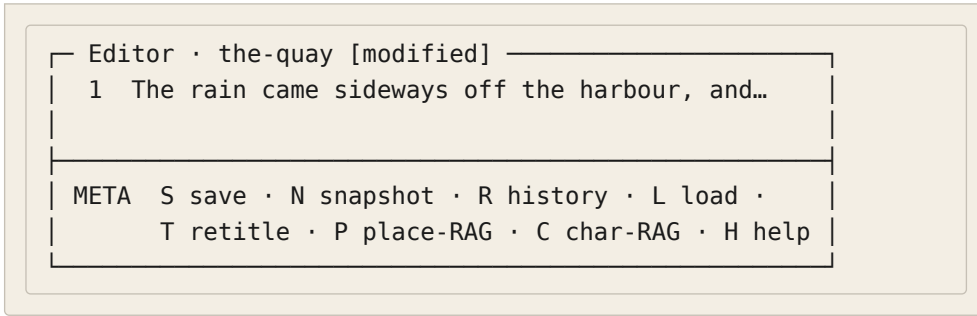
There are two prefixes, and the difference between them is a genuine distinction of **kind**, not an accident of which keys were free.

Ctrl+B — the meta prefix

`Ctrl+B` is the **meta** prefix: it introduces commands that **act on your book** — add a chapter, take a snapshot, run a check, tag a paragraph, translate a line. Press `Ctrl+B` and release it, and the tool enters **meta mode**: the status bar lights its yellow **META** chip and prints the actions available for the pane you are in. The next key you press selects one of them. `Esc` backs out of a pending chord without doing anything, and any key the table does not recognise cancels with a hint naming which pane's table it consulted.

> TERM **Meta prefix**
>
> `Ctrl+B` — the key that opens **meta mode**. The single most important thing to learn about it is that the action table is **pane-specific**: `Ctrl+B S` means **save the paragraph** in the Editor, **add a subchapter** in the Tree, and **clear the inference** is one of the AI pane's few actions. The same second key, read against a different pane, is a different command.

● ● ● *Meta mode pending — the status bar prompts for the second key*



That the table is pane-specific is not a complication to memorise but a simplification to lean on. You do not learn one enormous list of chords; you learn the handful that belong to the pane you are working in, and the same letters mean the natural thing in each place. In the Tree, `Ctrl+B B`, `C`, `S`, `P` add a **book**, a **chapter**, a **subchapter**, and a **paragraph** — the second key is the word's first letter. In the Editor, `Ctrl+B N` takes a **new** snapshot and `Ctrl+B R` opens its **history**. A last tier of meta chords — the ones that run whole-book intelligences, like `Ctrl+B Shift+C` to run every fast checker at once — work from **any** pane, because they belong to the book rather than to a place in it.

Ctrl+V — the view prefix

`Ctrl+V` is the **view** prefix, and it introduces a different kind of command: things that **show you a view** or **reach a tool** rather than change the manuscript's structure — export this paragraph to markdown, open the fuzzy paragraph picker, float a rendered preview, jump a bookmark, run the Editorial Pass, open the command palette. Where `Ctrl+B` is about **acting on the book**, `Ctrl+V` is about **looking through it** and reaching the pickers and panels that help you look. It reads the same way — press and release `Ctrl+V`, then the action key — and like the meta layer it is fully rebindable.

The mnemonic is worth stating out loud, because it is what lets you **guess** a chord correctly instead of looking it up: `Ctrl+B` for the **book** (things you do to it), `Ctrl+V` for the **view** (things you do to see it). Nearly every chord you meet in this manual sits under one of these two prefixes, and the prefix tells you which half of your mind the command lives in.

Overlays — the modal stack on top of it all

Some chords do not run and return; they **open something** — a picker, a confirmation, a full-screen editor for your config, a floating preview. These are **overlays**, and while one is up it sits on top of the panes and takes your keys for itself. The pane beneath keeps its visual focus but does not see input until the overlay closes, and `Esc` is the near-universal way to close the top overlay and drop back to what was underneath. Overlays stack: a picker can open a confirm on top of itself, and closing the confirm returns you to the picker. This is why `Esc` is the key you can always press when you feel lost — it peels one layer off the top and never does anything destructive.

The status bar and the title bars

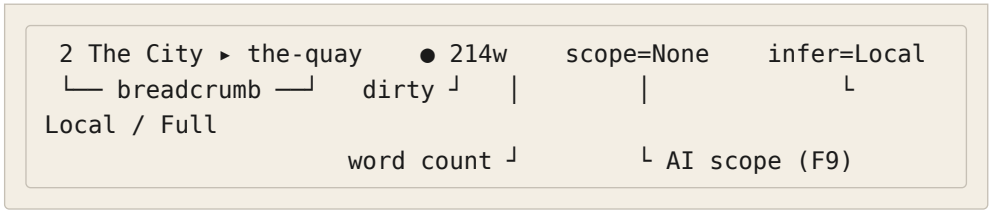
The bottom line and the little labels along the tops of panes are not decoration; they are a continuous, quiet readout of state. Learning to glance at them is most of what "knowing where you are" amounts to.

The status bar

The status bar is the window's single most information-dense line. In calm moments it shows the breadcrumb path to your cursor, the open paragraph's word count, a red ● when there are unsaved edits, and the current AI scope and inference mode. In busy moments it is also where transient messages land — a search reporting `match 1 / N`, a diagnostic jump reading `diag 2/5 line 40:12`, a `marked N`

count while you multi-select in the Tree, the yellow **META** prompt while a chord is pending, or a notice that a file changed on disk and was reloaded. It is the first place to look when you wonder what just happened.

● ● ● *The status bar — a running readout of the moment*



Optional chips ride the same line when you turn them on — a POV chip naming the paragraph's viewpoint character, a `lang=` chip for the prompt language, a live syllable readout while a verse paragraph is open. They are quiet by default and each has its own toggle; the point is that the status bar is the one place Inkhaven speaks to you without being asked.

The title bars

Every pane wears a thin title along its top, and each title earns its space. The Editor's names the open paragraph and appends `[modified]` when it is dirty. The Tree's is simply the pane's name. The right region's title tells you which of the three panes you are looking at and its live state — the AI pane's reads like `AI · llama · done · infer=Full · scope=Paragraph`, folding in the provider, the stream state, the inference mode, and the scope; the Output pane's shows a `shown/total · filter` count; the AI prompt bar's title echoes the scope as `AI prompt · scope: Paragraph`. You rarely read them word for word, but they are the reason you are never guessing which mode a pane is in.

Finding a chord — the palette and the quick reference

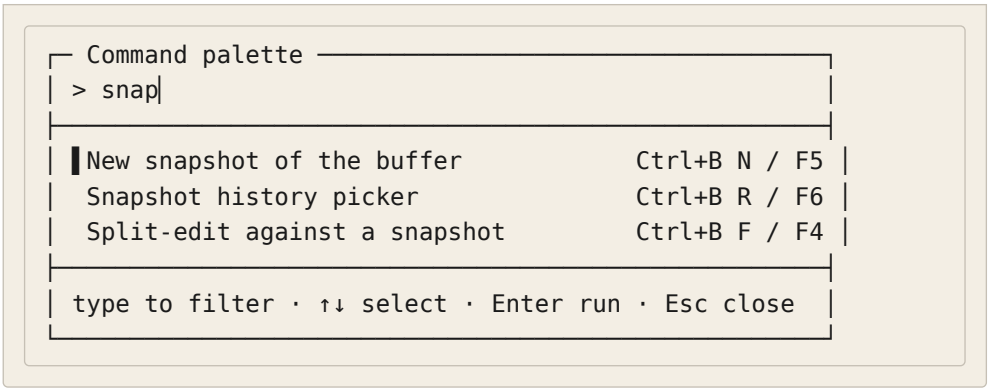
Two prefixes and a pane-specific table is a system you can **reason** about, but nobody memorises every chord, and Inkhaven does not ask you to. Three surfaces exist precisely so you can find a command without remembering its keys.

The command palette — Ctrl+V Space

`Ctrl+V Space` opens the **command palette**: a fuzzy search over the entire keybind registry. Start typing any part of a command's name, its chord, or its description, and the list narrows to match; `↑↓` moves the selection, `Enter` runs it,

`Esc` closes. This is the fast way to reach **any** command without knowing its chord — type "snapshot", or "fact", or "outline", and run what comes up. It is also how you **learn** the chords: the palette shows each command's key beside its name, so the thing you searched for by word today you will strike by chord tomorrow.

● ● ● *Ctrl+V Space — fuzzy-find any command in the registry*



The quick reference — Ctrl+B H, and ? from the Tree

Where the palette is for **running** a command you can half-name, the **quick reference** is for **browsing** what is available from where you stand. `Ctrl+B H` opens it from any pane — a floating, scrollable panel of the chords that apply, pane-aware so it shows you the ones that matter here first. Scroll with the arrows or `PgUp` / `PgDn`; `Esc` closes it. When the Tree has focus you can also open it with a bare `?`, since the Tree is the one pane where `?` is not a character you would type; in the Editor and the input bars `?` stays a literal question mark, and `Ctrl+B H` is the way in.

Ctrl+V Space	Command palette — fuzzy-find and run any command by name, chord, or description.
Ctrl+B H	Quick reference — the pane-aware chord panel, from any pane.
?	Quick reference, but only when the Tree pane has focus.
Ctrl+B V	Version / credits pane — the running version, author, licence, and dependencies.

Between them the two surfaces cover both ways a person forgets a chord. When you know **what you want** but not the keys, reach for the palette. When you want

to **see what is possible** from where you are, open the quick reference. Neither asks you to leave the window or read this book.

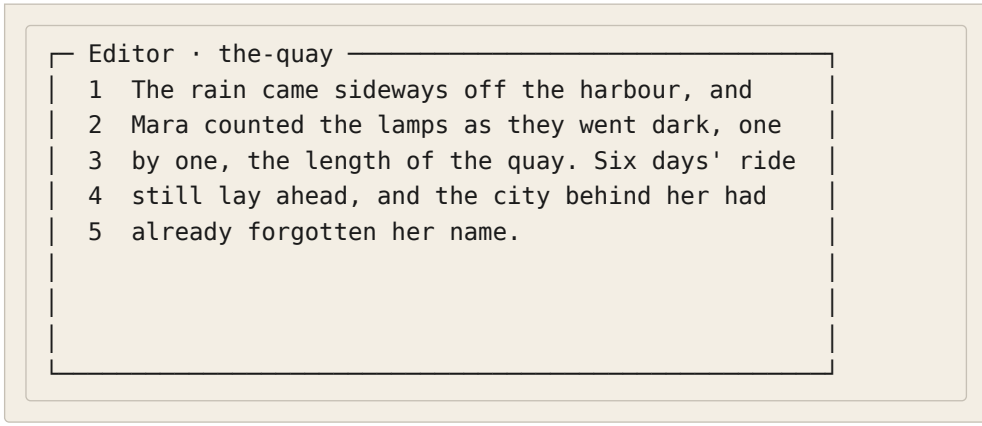
Fullscreen and focus modes

The four-pane layout is the working default, but sometimes you want less on the screen — the prose alone while you draft, or one right pane blown up while you read a long result. Inkhaven has a mode for each, and each is a toggle: press to enter, press again to return.

Focus mode — Ctrl+B Shift+W

`Ctrl+B Shift+W` enters **focus mode** — the distraction-free layout. It hides the Tree, the AI pane, the Search bar, and the AI prompt, and gives the whole window to the Editor, forcing focus there as it opens. It is the mode for the moment you have decided to stop tending the book and simply write it: nothing on screen but your paragraph. Press `Ctrl+B Shift+W` again to restore the four panes. (The setting is called "typewriter mode" in a few older strings and in the config file, for backward compatibility — same mode, the friendlier name won.)

● ● ● *Focus mode — the Editor alone, everything else hidden*



Fullscreen panes — Ctrl+Z f and Ctrl+B K

When it is a **result** you want to sink into rather than the prose, fullscreen the right pane instead. `Ctrl+Z f` blows up the currently shown right pane — the Output pane or the Thoughts pane — to fill the window, for reading a long notice board or a developmental brief without the Editor crowding it. The AI pane has its own dedicated fullscreen, `Ctrl+B K`, which lays out the AI pane, its scrollable chat

history, and the prompt across the whole window for a long conversation. Focus mode and AI-fullscreen are mutually exclusive — you are either writing alone or talking at length, not both — and each toggle returns you to the four-pane layout.

Ctrl+B Shift+W	Focus mode — hide everything but the Editor. Press again to restore.
Ctrl+Z f	Fullscreen the current right pane (Output / Thoughts).
Ctrl+B K	Fullscreen the AI pane with its chat history and prompt.

A note on the mouse

Inkhaven is a keyboard instrument, but it does not ignore the mouse. It captures mouse input on start: a left-click moves focus to the pane you click, and inside a pane the click lands where you expect — the row cursor in the Tree, the character cursor in the Editor (clicks in the number gutter are ignored). The scroll wheel scrolls whatever it is over — three rows per tick in the Tree, three lines in the Editor, the chat history in the AI pane, the message list in the Output pane. It is enough to point and scroll when a hand is already on the mouse, and it is entirely optional — nothing in this book requires it.

> **SELECTING TEXT THROUGH THE TERMINAL**
>
> Because Inkhaven captures the mouse, dragging to select does not reach your terminal's own copy by default. Hold `Shift` (or `Option`, depending on the terminal) while you drag to bypass the capture and select through the terminal, or toggle capture off entirely with `Ctrl+Shift+M` when you want the terminal's native clipboard for a while.

The window you have just mapped does not change from here. Every later chapter — the editor's depths, the AI's scopes, the world and its facts, the reading intelligences — plays out on these same four panes, reached by these same chords, read off this same status bar. You will spend the rest of the book learning what to **do** in this window; you now know what the window **is**.

WHAT YOU LEARNED

- The window is **four regions**: the **Tree** (left), the **Editor** (centre), and a **right slot** that shows one of the **AI**, **Output**, or **Thoughts** panes, over a Search bar, an AI prompt, and a status bar.
- Plain `Tab` cycles focus **Tree → Editor → current right pane**; `Ctrl+B Tab` cycles **which** pane the right slot shows (**Output → AI → Thoughts**); `Ctrl+T`, `Ctrl+1`, and `Ctrl+3` jump straight to a pane.
- A **chord** is a two-key command: `Ctrl+B` (the **meta** prefix — **act on the book**) or `Ctrl+V` (the **view** prefix — **reach a view or tool**), then a **pane-specific** action key. `Esc` cancels a pending chord or closes the top overlay.
- The **status bar** and **title bars** are a running readout — breadcrumb, word count, the red ● dirty chip, AI scope and inference mode, and the yellow **META** prompt while a chord waits.
- You never have to memorise a chord: `Ctrl+V Space` fuzzy-finds and runs any command, `Ctrl+B H` (or `?` in the Tree) opens the pane-aware quick reference.
- `Ctrl+B Shift+W` gives the window to the Editor alone; `Ctrl+Z f` fullscreens the current right pane and `Ctrl+B K` the AI pane — each a toggle back to the four panes.

PART II

Writing

CHAPTER 4

4

The Editor

Moving, selecting, finding and replacing, undo and redo, and the split-edit view — the day-to-day mechanics of putting words on the page.

CHAPTER 5

5

The Tree and Its Files

Building and reshaping the manuscript tree, and the leaf types that live in it: prose, HJSON data, Jinja templates, Bund scripts, images, and the structural paragraph subtypes.

CHAPTER 6

6

Snapshots and History

Every paragraph's versioned history: taking and restoring F6 snapshots, the project-wide snapshot browser, and recovering deleted prose.

CHAPTER 7

7

Search

Finding any passage two ways — full-text and semantic (meaning-based, multilingual) — and how the local embedding index finds a paragraph that never used your exact words.

CHAPTER 8

8

The Style Overlays

The in-editor coaching overlays — filter-word crutches, echoed words, telling-not-showing, the POV chip, the sentence-rhythm gauge — each a toggle you question rather than obey.

PART III

The AI Assistant

CHAPTER 9

9

The AI Assistant

The AI pane end to end: choosing the scope, the local-versus-full knowledge mode, where the answer lands, and how six providers (and any the router reaches) plug in.

CHAPTER 10

10

Chat With Your Book

Retrieval-augmented conversation grounded in your own manuscript — the Book scope and the F9 structured scopes (Facts, Graph, and the reader conversations).

CHAPTER 11

11

Reusable Prompts

Writing prompt templates once and reusing them: the `prompts.hjson` library, the project-local Prompts book, the substitutions, and the picker.

CHAPTER 12

12

Watching the Cost

The cost dashboard and the daily caps — how Inkhaven keeps model spend visible and informs rather than surprises, and what each cost-capped feature actually costs.

PART IV

The World & Its Facts

CHAPTER 13

13

Places and Characters

The Places and Characters system books: recording your world's people and locations, the in-editor highlight overlays, and asking the AI about any of them.

CHAPTER 14

14

The World

Declaring a world's physics in `world.hjson` and letting the deterministic compiler derive its astronomy, geology, climate and demographics — plus maps and the live fact-checker.

CHAPTER 15

15

The Knowledge Graph

The typed-edge semantic net over your nodes, how it is populated, and chatting with it — walking the graph to answer a plain question about your own book.

CHAPTER 16

16

The Timeline

Recording events on a calendar of your own devising, and the timeline critique that catches orphans and impossible overlaps.

PART V

The Intelligences

CHAPTER 17

17

Continuity and Knowledge

The two continuity intelligences that watch a manuscript: SENTINEL (the book watches itself) and KEN (who knows what, when). How to run each and read its ledger.

CHAPTER 18

18

The Read-Through and the Voices

LECTOR reads the book forward as a first reader; CHORUS profiles the cast's voices; the dialogue and narrator-voice readers round out how it sounds.

CHAPTER 19

19

Revision and History

Turning findings into confirmed changes with REDLINE's Editorial Pass, and measuring whether a revision helped with CHRONICLE's draft history.

CHAPTER 20

20

The Inner Family

The readers who question rather than answer — Inner Socrates, Editor, Theologian, Poet, and the reasoning-rigor reader — and how to engage each.

PART VI

Language, Verse & Scholarship

CHAPTER 21

21

Constructed Languages

Building an invented language inside the editor — phonology, lexicon, morphology — and the WordNet thesaurus for real-world prose.

CHAPTER 22

22

Poetry

Measuring verse against a declared form — metre, rhyme, syllables — with a reader that observes and never generates.

CHAPTER 23

23

Research and Scholarship

Grounding a book in fact: the Research Assistant, the Sources bibliography, terminology governance, and the scholarly apparatus (loci, verborum, argument outline).

PART VII

Producing the Book

CHAPTER 24

24

Assembly and PDF

Turning the tree into a Typst-compilable directory and a finished PDF — the two-chord path from buffer to book.

CHAPTER 25

25

EPUB, Web and More

The other destinations: EPUB, an HTML website, the arXiv scientific bundle, DOCX, and the audiobook.

CHAPTER 26

26

Technical Documentation

The features for technical and reference books: the `docs` verification checks and the back-of-book index.

PART VIII

Scripting with Bund

CHAPTER 27

27

Scripting with Bund

The embedded scripting language: hook lambdas, the `ink.*` standard library, Bund Script nodes, and the sandbox policy that keeps them safe.

PART IX

Keeping It Healthy

CHAPTER 28

28

Keeping It Healthy

Backups and restore, the `reindex` command, the project doctor, and the advisory project lock — keeping the store sound over years of use.

CHAPTER 29

29

Configuration

A tour of `inkhaven.hjson` — how the layered config works, the blocks you are most likely to touch, and how to edit it safely from inside the editor.

APPENDIX A

A

The Keybinding Map

Every chord the TUI honours, by pane and overlay — the printable reference. Mirrors the canonical `KEYBINDING.md`.

APPENDIX B

B

The Command Line

Every `inkhaven` subcommand at a glance, grouped by what it does.

APPENDIX C

C

The Configuration Reference

Every `inkhaven.hjson` block and field, with its default. Mirrors the canonical `CONFIGURATION.md`.

APPENDIX D

D

The Feature Index

The canonical map — every feature to its command, chord, doc, and scripting namespace.

AFTERWORD

About the author

Vladimir Ulogov.

Vladimir Ulogov has spent decades building infrastructure for distributed systems — the kind of software that watches other software. Early in his career he worked on monitoring and telemetry platforms; later years took him into federated observability, telemetry buses, and the architecture of systems that have to make sense of millions of data points without losing the thread.

Observability, in the end, is the art of **holding a system too large for one head** — of watching it without losing the thread, and never reporting a thing it cannot back up. It is not an accident that a tool he built for writers carries the same instinct to the desk: a book, past a certain size, is exactly such a system, and it too

> deserves an instrument that holds it for you.

What makes him slightly unusual in his corner of the industry is a tendency to write his own tools — not in the sense of small utilities, but in the sense of programming languages. The Bund language (its compiler, its VM, its document store, its parser) lives in a long series of Rust crates on crates.io. `rust_dynamic`, `rust_multistack`, `rust_multistackvm`, `bundcore`, `zbus_universal_data_gateway` — each is a building block that exists because the off-the-shelf options didn't fit the shape of the work.

Why Inkhaven exists

Inkhaven is Vladimir's personal reflection on how a single tool can hold the **whole** of writing a book. The frustration it answers is one every long-form writer knows: that the work is scattered across a dozen apps that don't talk to each other — a browser for research, one program for the prose, another for the characters, a spreadsheet for the word count, a design tool for the final typesetting — and every seam between them is a place the book leaks.

Inkhaven's wager is that all of it belongs in one place, next to the keyboard: the prose and the world it is set in, the facts it must not contradict, the assistant you steer, and the finished PDF. Not because integration is elegant, but because a book past a certain size outgrows the mind writing it, and a tool that holds your words should also be able to **hold the book** — its facts, its continuity, its shape — so the writer is free to do the one thing no tool can, which is write.

A work of love

Inkhaven is open source. The licence is permissive — you can read it, fork it, study it, modify it, and pass it on. Strictly speaking, the licence also lets you sell it. The author would, gently but firmly, disagree with you doing that. Inkhaven was not designed as a *for sale* project. It is a work of love made for the authors who can least afford to pay for software, and turning it into a commercial product would betray the reason it exists. So please — don't.

It carries no analytics, no telemetry, no upsell. The binary will never phone home; the project will never have an "Enterprise" tier you have to escape from.

This was a deliberate choice. There are excellent commercial tools for writing. They cost money — which is fine for many writers, and a barrier for many more. Inkhaven exists for the second group: for the graduate student writing a dissertation on a battered laptop, for the novelist who shouldn't have to pick between rent and software, for the engineer drafting in the same terminal where they already write code, for anyone who would benefit from a tool that respects their work without asking for a credit-card number.

A note on cooperation

Vladimir believes firmly in the human capacity for mutual help — that we make better work, and live better lives, when we share what we know and what we build. Open source is one of the most concrete expressions of cooperation our era has produced: code read, improved, and passed forward without payment, without permission, by people who will never meet.

If Inkhaven helps you finish your book — that is enough. If it gives you a chord pattern you adapt into your own tool — that's a gift back to the larger project of making software writers can love. As a society, we achieve the greatest things when we help each other rather than compete with each other.

Where to find more

GitHub	@vulogov — the source for Inkhaven, Bund, and the dozen-plus Rust crates that carry the infrastructure. Issues and PRs welcome.
LinkedIn	/in/vladimirulogov — posts on observability, the occasional long-form essay.
YouTube	@vulogov — talks and walkthroughs from the conference trail.
X / Twitter	@vladimir_ulogov

If a fact you grounded ever surprises you, or a command trips you up and you can't find the answer in this book — open an issue on GitHub. The author reads them.

END OF THE BOOK · INKHAVEN 3.0.0